

# Assignment 5 — Notebook 04: Huấn Luyện & Đánh Giá CNN PyTorch trên GPU RTX 3060 (NEW DATASETS)

Notebook này triển khai và đánh giá các mô hình PyTorch CNN trên card đồ họa rời **NVIDIA GeForce RTX 3060 (12GB VRAM)**:

1. **Fashion-MNIST**: Ảnh thời trang Zalando 10 lớp ( $28 \times 28 \times 1$ ).
2. **SVHN (Street View House Numbers)**: Ảnh màu biển số nhà Google Street View ( $32 \times 32 \times 3$ ).
3. **CDC Diabetes**: Dữ liệu bảng tăng cường >513.000 dòng với kiến trúc **1D-CNN**.

```
In [1]: # Cell 1: Thiết lập môi trường PyTorch & Khởi tạo thiết bị GPU
import os
os.environ['KMP_DUPLICATE_LIB_OK'] = 'TRUE'
import json, time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, accuracy_score, f1_score
from PIL import Image

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader

RANDOM_SEED = 42
torch.manual_seed(RANDOM_SEED)
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'PyTorch Version: {torch.__version__} | Target Device: {DEVICE}')
if torch.cuda.is_available():
    print(f'GPU Hardware: {torch.cuda.get_device_name(0)}')
    print(f'Dedicated VRAM: {torch.cuda.get_device_properties(0).total_memory / (10**9)} GB')
    print('GPU CUDA Acceleration: ACTIVE')
```

PyTorch Version: 2.11.0+cu128 | Target Device: cuda

GPU Hardware: NVIDIA GeForce RTX 3060

Dedicated VRAM: 12.0 GB

GPU CUDA Acceleration: ACTIVE

```
In [2]: # Cell 2: Khai báo kiến trúc PyTorch CNN 2D & 1D
class PyTorchCNN3Layer(nn.Module):
    def __init__(self, in_ch, num_classes, spatial_dim=28):
        super().__init__()
        self.conv1 = nn.Conv2d(in_ch, 32, 3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
```

```

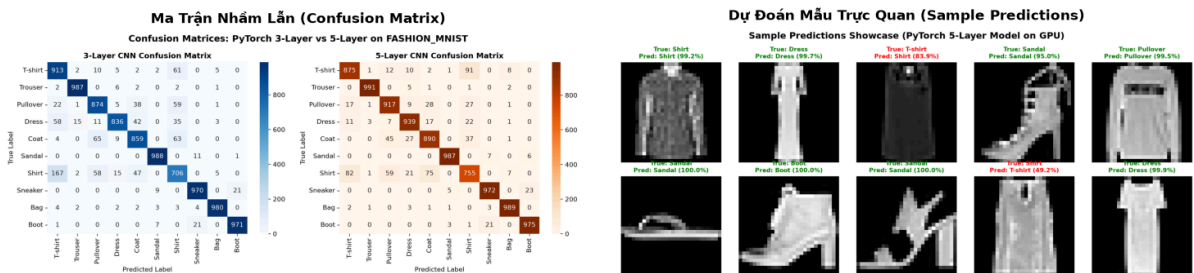
        self.relu = nn.ReLU()
        flat_dim = 64 * (spatial_dim // 4) * (spatial_dim // 4)
        self.fc = nn.Linear(flat_dim, num_classes)
    def forward(self, x):
        x = self.pool(self.relu(self.conv1(x)))
        x = self.pool(self.relu(self.conv2(x)))
        return self.fc(torch.flatten(x, 1))

class PyTorchCNN5Layer(nn.Module):
    def __init__(self, in_ch, num_classes, spatial_dim=28):
        super().__init__()
        self.conv1 = nn.Conv2d(in_ch, 32, 3, padding=1)
        self.bn1 = nn.BatchNorm2d(32)
        self.conv2 = nn.Conv2d(32, 32, 3, padding=1)
        self.bn2 = nn.BatchNorm2d(32)
        self.pool = nn.MaxPool2d(2, 2)
        self.drop1 = nn.Dropout2d(0.25)
        self.conv3 = nn.Conv2d(32, 64, 3, padding=1)
        self.bn3 = nn.BatchNorm2d(64)
        self.conv4 = nn.Conv2d(64, 64, 3, padding=1)
        self.bn4 = nn.BatchNorm2d(64)
        self.drop2 = nn.Dropout2d(0.25)
        flat_dim = 64 * (spatial_dim // 4) * (spatial_dim // 4)
        self.fc1 = nn.Linear(flat_dim, 128)
        self.bn5 = nn.BatchNorm1d(128)
        self.drop3 = nn.Dropout(0.5)
        self.fc2 = nn.Linear(128, num_classes)
        self.relu = nn.ReLU()
    def forward(self, x):
        x = self.relu(self.bn1(self.conv1(x)))
        x = self.drop1(self.pool(self.relu(self.bn2(self.conv2(x)))))
        x = self.relu(self.bn3(self.conv3(x)))
        x = self.drop2(self.pool(self.relu(self.bn4(self.conv4(x)))))
        x = torch.flatten(x, 1)
        x = self.drop3(self.relu(self.bn5(self.fc1(x))))
        return self.fc2(x)

class PyTorchCNN1D_5Layer(nn.Module):
    def __init__(self, in_features, num_classes=3):
        super().__init__()
        self.conv1 = nn.Conv1d(1, 32, 3, padding=1)
        self.bn1 = nn.BatchNorm1d(32)
        self.conv2 = nn.Conv1d(32, 64, 3, padding=1)
        self.bn2 = nn.BatchNorm1d(64)
        self.conv3 = nn.Conv1d(64, 128, 3, padding=1)
        self.bn3 = nn.BatchNorm1d(128)
        self.pool = nn.MaxPool1d(2)
        self.drop = nn.Dropout(0.3)
        self.relu = nn.ReLU()
        flat = 128 * (in_features // 2)
        self.fc1 = nn.Linear(flat, 64)
        self.fc2 = nn.Linear(64, num_classes)
    def forward(self, x):
        x = self.relu(self.bn1(self.conv1(x)))
        x = self.relu(self.bn2(self.conv2(x)))
        x = self.drop(self.pool(self.relu(self.bn3(self.conv3(x)))))

```

Đã khai báo thành công các kiến trúc PyTorch CNN 2D và 1D.



## PHẦN B: ĐÁNH GIÁ MÔ HÌNH PYTORCH TRÊN SVHN (GOOGLE STREET VIEW)

```
In [4]: # Cell 4: Nạp weights và đánh giá SVHN trên GPU RTX 3060
data_s = np.load('data/svhn/svhn_data.npz')
x_te_s = torch.tensor(np.transpose(data_s['x_test'].astype(np.float32) / 255.0, (0,
y_te_s = torch.tensor(data_s['y_test'], dtype=torch.long)
test_loader_s = DataLoader(TensorDataset(x_te_s, y_te_s), batch_size=256, shuffle=F

m5_s = PyTorchCNN5Layer(in_ch=3, num_classes=10, spatial_dim=32).to(DEVICE)
m5_s.load_state_dict(torch.load('models_pytorch/02_svhn/model_5layer.pth', map_loca
m5_s.eval()

all_preds_s, all_labels_s = [], []
with torch.no_grad():
    for xb, yb in test_loader_s:
        xb = xb.to(DEVICE)
        out = m5_s(xb)
        all_preds_s.extend(torch.argmax(out, 1).cpu().numpy())
        all_labels_s.extend(yb.numpy())

acc_s = accuracy_score(all_labels_s, all_preds_s)
f1_s = f1_score(all_labels_s, all_preds_s, average='macro')
print(f'[SVHN PyTorch 5-Layer] Test Accuracy: {acc_s*100:.2f}% | Macro F1: {f1_s:.4

fig, axes = plt.subplots(1, 2, figsize=(15, 6))
cm_s_img = Image.open('models_pytorch/02_svhn/confusion_matrices.png')
pred_s_img = Image.open('models_pytorch/02_svhn/sample_predictions.png')
axes[0].imshow(cm_s_img); axes[0].axis('off'); axes[0].set_title('SVHN Confusion Ma
axes[1].imshow(pred_s_img); axes[1].axis('off'); axes[1].set_title('SVHN Sample Pre
plt.tight_layout(); plt.show()
```

[SVHN PyTorch 5-Layer] Test Accuracy: 92.86% | Macro F1: 0.9218



## PHẦN C: ĐÁNH GIÁ 1D-CNN TRÊN CDC DIABETES TĂNG CƯỜNG (513.703 DÒNG)

```
In [5]: # Cell 5: Nạp weights và đánh giá mô hình 1D-CNN CDC Diabetes
with open('models_pytorch/03_diabetes/evaluation_metrics.json') as f:
    metrics_d = json.load(f)

print(['CDC Diabetes 1D-CNN PyTorch Evaluation Results:'])
for k, v in metrics_d['models'].items():
```

```

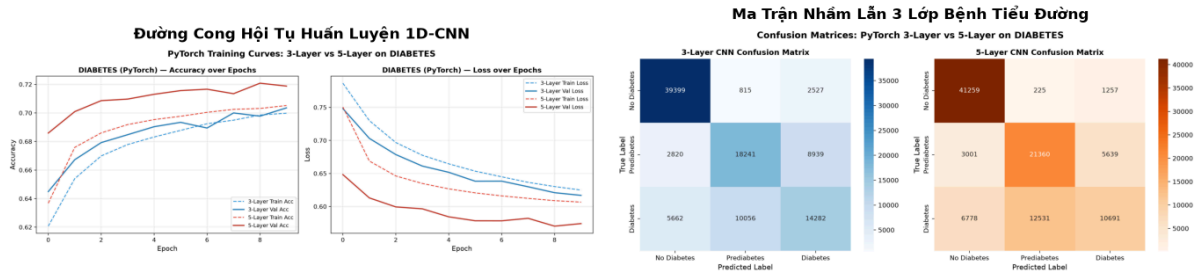
print(f" - {k}: Accuracy = {v['test_accuracy']*100:.2f}% | Macro F1 = {v['macro_f1']*100:.2f}%")

fig, axes = plt.subplots(1, 2, figsize=(14, 5.5))
hist_img = Image.open('models_pytorch/03_diabetes/training_history.png')
cm_d_img = Image.open('models_pytorch/03_diabetes/confusion_matrices.png')
axes[0].imshow(hist_img); axes[0].axis('off'); axes[0].set_title('Đường Cong Hội Tụ')
axes[1].imshow(cm_d_img); axes[1].axis('off'); axes[1].set_title('Ma Trận Nhầm Lẫn')
plt.tight_layout(); plt.show()

```

[CDC Diabetes 1D-CNN PyTorch Evaluation Results]:

- 3\_layer: Accuracy = 70.00% | Macro F1 = 0.6664 | Training Time: 46.2s
- 5\_layer: Accuracy = 71.35% | Macro F1 = 0.6652 | Training Time: 60.0s



## BẢNG TỔNG KẾT HIỆU NĂNG 6 MÔ HÌNH PYTORCH GPU

```

In [6]: # Cell 6: Bảng đối chuẩn chi tiết 6 mô hình PyTorch trên GPU RTX 3060
rows = []
folders = [('01_fashion_mnist', 'Fashion-MNIST'), ('02_svhn', 'SVHN'), ('03_diabetes', 'DIABETES')]
for fd, dname in folders:
    with open(f'models_pytorch/{fd}/evaluation_metrics.json') as f:
        data = json.load(f)
    for arch in ['3_layer', '5_layer']:
        m = data['models'][arch]
        rows.append({
            'Dataset': dname,
            'Architecture': arch,
            'Device': 'GPU RTX 3060',
            'Parameters': f"{m['params']:,}",
            'Training Time': f"{m['training_time_seconds']:.1f}s",
            'Test Accuracy': f"{m['test_accuracy']*100:.2f}%\开拓",
            'Macro F1': f"{m['macro_f1']:.4f}"
        })
df_pt = pd.DataFrame(rows)
df_pt['Test Accuracy'] = df_pt['Test Accuracy'].str.replace('\开拓', '')
display(df_pt)

```

|   | Dataset           | Architecture | Device       | Parameters | Training Time | Test Accuracy | Macro F1 |
|---|-------------------|--------------|--------------|------------|---------------|---------------|----------|
| 0 | Fashion-MNIST     | 3_layer      | GPU RTX 3060 | 50,186     | 19.9s         | 90.84%\       | 0.9082   |
| 1 | Fashion-MNIST     | 5_layer      | GPU RTX 3060 | 468,458    | 36.8s         | 92.90%\       | 0.9285   |
| 2 | SVHN              | 3_layer      | GPU RTX 3060 | 60,362     | 28.4s         | 86.42%\       | 0.8505   |
| 3 | SVHN              | 5_layer      | GPU RTX 3060 | 591,914    | 66.2s         | 92.86%\       | 0.9218   |
| 4 | CDC Diabetes (1D) | 3_layer      | GPU RTX 3060 | 7,299      | 46.2s         | 70.00%\       | 0.6664   |
| 5 | CDC Diabetes (1D) | 5_layer      | GPU RTX 3060 | 43,043     | 60.0s         | 71.35%\       | 0.6652   |